wrt variable because g want to access Read and write in mutual exclusive way. (synchronized way)

③ Read Count :- this variable ~~get~~ initialize to zero tracks how many Reader are reading in CS

Writer Pro

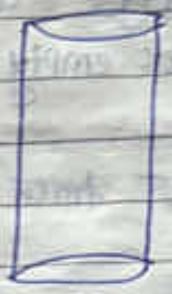
```
do
{
 wait(wrt);
 // do write operations
  signal(wrt);
} while(true);
```

wait(wrt) → when we write wait lock so that another thread not coming.
than Perform write operation than get signal to Enter any thread.

Reader-writer Problem

① Reader thread :- Read data
② writer thread :- write data

we want
only one writer but multiple Reader can Read data.



Problem when
>1 writer OR 1 writer and some other thread Read or write Parrlely than Race condition occurs. and gn data base data may be gnconsistence.

[when writer write and at the same time Reader Read data. and after some time writer Update data than Reader not Read correct data.]

our am gs to only one writer at the same time.

→ semaphores soln to R-W Problem
① mutex :- binary semaphore
to ensure mutual exclusion, when Read Count(rc) gs Updated.
→ no two thread modify R.C at the same time
we now more than one thread can Read at same time. so we maintain Read count gt gs
② wrt → binary semaphore semaphore

Page No.:
Date: / /

Soln by semaphores

① m, mutex :- binary semaphores used to acqure lock on the buffer.

② empty :- a counting semaphores initial value as n track empty slots.

③ full :- track field slots initial=0.

Producer	Consumer

① do
{
② wait(empty)
//wait until empty > 0
② wait(mutex)
③ // as add data to buffer
④ signal (mutex);
⑤ signal (full); // increment
⑥ full value
⑦ } while(1)
	do
	{
	wait(full) //wait until full > 0
	then full--
	wait (mutex);
	//Remove data from buffer
	signal (mutex);
	signal (empty); // increment empty
	} while(1)

initially producer inside C.S. because empty=0 now add data to buffer. now signal to mutex and increment mutex = 1
when increment = 1 wait(full) call from consumer and full-- and Remove data from buffer now

S(2)

$T_1 \rightarrow$ wait() $\rightarrow$ s->val = 1

$T_2 \rightarrow$ wait() $\rightarrow$ s->val = 0

$T_3 \rightarrow$ wait() $\rightarrow$ s->val = -1 Block()

→ types of semaphore

→ lec-18

Producer Consumer Problem and it's solution :-
(bounded buffer Problem)

we have two thread
① Producer thread
② consumer thread

Producer thread produce data and put in the Buffer memory.

and consumer thread consume data.

→ in one time only one thread execute. either Producer Produce consumer Consume or consumer consume producer Produce.

→ Buffer memory is CS.

→

Problem :-

→ we want to access CS in synchronized way.

→ Producer must not insert data when the buffer is full.

→ consumer must not Remove/Pick data when the buffer is empty.

Page No.:
Date: / /

→ why to use conditional waiting?
to avoid busy waiting

→ contention is not here.

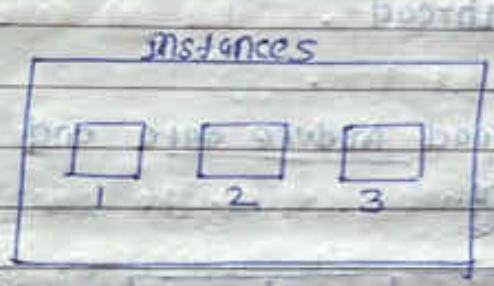
2. semaphores

→ synchronization method.

→ An integer that is Equal to no of Resources.

→ multiple thread can Go and execute C.S. concurrently.

$T_1 \rightarrow T_{10}$ instance



sem = ~~3~~ ~~2~~ ~~1~~ 0

T_1 Go to 1 $3-1=2$

T_2 Go to 2 Resource $2-1=1$

T_3 Go to 3 Resource $1-1=0$

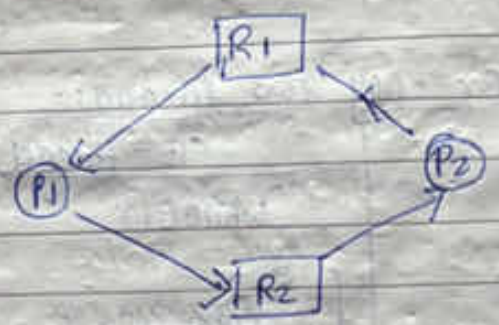
then all thread are lock until sem variable increment.

```
wait(S)
{
S->value --;
if (S->value <0)
{
add to S
Block();
}
}
```

```
signal(S)
{
S->value++
if (S->value <=0)
{
remove P from S->block
list.
WAKUP(P)
}
}
```

acquired the lock dies, then an other thread will be gofinite waiting.

② deadlock



P1 acc quire R1 Resource and P2 acc quire R2 Resource. P2 start execution when it have both R1 and R2 Resource. And P1 execute when it have both R1 and R2 in this situation deadlock will occurs.

③ Debugging (issue)

④ starvation

LEC-17

→ Conditional variable and semaphores

→ conditional variable method

thread can enter a wait state only when it has acquired a lock. when a thread Enter the wait state it will Relese the lock and wait until another thread notify that the event has occurred. once s the waiting thread Enter the Running thread it again acquires the locks immediatly and start executing.

gE's array [|]
0 1

Flag[2] → indicate if a thread is Ready to Enter the C.S

If Flag[i] = true. implies that P_i is Ready.

turn :- indicates whose turn is to Enter the C.S.
Value 0/1

→ Peterson's soln safe for only two thread :-

T_1 - thread	T_2 - thread
while(1)	while(1)
{	{
Flag[0] = T	Flag[1] = T
turn = 1	turn = 0
while(turn == 1 && Flag[1] = T)	while (turn == 0 && Flag[0] = T)
C.S	C.S
Flag[0] = F	Flag[1] = F
}	}

If for T_1 thread initially Flag[0] = T and turn = 1 than suppose context switch than Flag[1] = T and turn = 0 now T_2 Enter in C.S
mutual exclusion and Progress both satisfy.

→ Locks

ⓐ Lock can be use to implement mutual exclusion and avoid race condition by allowing only one thread/ Process to access critical section.

ⓑ Disadvantages

① Contention :- one thread has acquired the lock, other thread will be busy waiting. what if thread that had

5. Can we use a ~~simple~~ single flag variable to solve the problem of race condition?

(a) No.

E. Soln of C.S should have 3 conditions?

(1) mutual-exclusion

(2) Progress:- when C.S is idle than any thread can enter. suppose we have two thread T_1 and T_2. any thread can enter.

(3) Bounded waiting :- no indefinite waiting. Limited waiting time. (this is not mandatory)

$T = 0/1$

T_1 - thread	T_2 - thread
while(1)	while(1)
{	{
while (turn != 0)	while(turn != 1)
C.S	C.S
turn = 1	turn = 0
R.S	RS
}	}

If $T = 0$ $T_1 \rightarrow T_2$ (first T_1 schedule than T_2)

If $T = 1$ $T_2 \rightarrow T_1$ (first T_2 schedule than T_1)

If turn = 0 and 0 != 0 condition false.

(1) mutual exclusion satisfy

(2) Progress not satisfy.

turn = and go in C.S.

on other side 0 != 1 condition true so wait for CS

→ Peterson soln :-

now turn = 1, so on the side of T_2 1 != 1 condition false. so enter in CS

Page No.:
Date: / /

① Race condition :-

[A race condition occurs when two or more threads can access shared data and they try to change it at the same time, because thread scheduling algorithm can swap b/w thread at any time.]
you don't know the order in which the thread will attempt to access the shared data.
therefore result of the change in data is dependent on the thread scheduling algorithm.
ie both thread are racing to change data.

④ solution to Race condition

ⓐ atomic operations :- make critical code section an atomic operations.

Ex :- execute in one CPU cycle.
Process इसे तरह बनाओ कि वह devide न हो।
C++ is thread safe. thread safe means context switch when this thread is completly executed.

ⓑ mutual exclusion :-

$T_1 \longrightarrow T_2$

sequentially executed T_1 finished execution then T_2 start.

ⓒ semaphores

A, B, C, D are Aadhar centre. and Admin count
How many total Aadhar card is make till now.
A, B, C, D सारे एक ही time पर बोला count++
कर दो Admin database are shared Resource.
A, B, C, D are thread.

suppose count = 11
count++
temp = count + 1
temp = 12

at the same time context switch to B thread
but count is not updated till now.
B → count + 1
temp = 11 + 1 = 12
now again context switch to A. and update count
now count = 12
here data are inconsistent. this is raise condition.

① Process synchronization technique play a role in
maintaining the consistency of data.

② critical section (C.S)
critical section is a segment of code, where
Process/threads access shared Resource such as
common variable or files, and perform write operation
on them.
Since Process/thread execute concurrently
any Process can be interrupted mid execution.

③ Major thread scheduling issue :-

Page No.:
Date: / /

Communicate with Each other and share resource because use same memory address space.

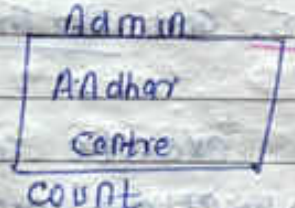
→ More Economy :-
It's more economical to create and context switch threads. also allocating memory and Resource for Process creation is costly so better to devide tasks into thread of same Process.

→ thread allow utilization of multiprocessor architectures to Create scale and efficiency.

LEC-16

~~you write count = count + 1~~
you write count++ it means count = count + 1
but internally temp = count + 1 and count = temp is done.

Example :-



→ CPU cache state is preserved.

⊙ How each thread get access to the CPU?
→ Each thread has its own programe counter and depending upon the thread scheduling algo. os schedule these thread.
→ OS will fetch instruction corresponding to PC. of that thread and execute instruction.

⊙ I/O or TQ based context switching is done here as well.

⊙ multithreading achive by one CPU.
① Never
② As two threads have to context switch for that single CPU.
suppose P_1 Process. we devide in thread t_1 and t_2 but we have only one CPU. so CPU get time to t_1 than context switch and get time to t_2 so we don't have benifits.

⊙ benifits of multithreading :-
① Responsiveness.
② Resource sharing :-
two Process communicate with each other via IPS. it may be share memory. or Pipeline it's more overhead. and Resource sharing b/w two Process is more overhead.
but thread is a part of Process. so it easily

Page No.:
Date: / /

Lec-15 Introduction to Concurrency

① Concurrency :- is the execution of the multiple instruction at the same time.
it happens in the OS when there are several process threads running in parallel.

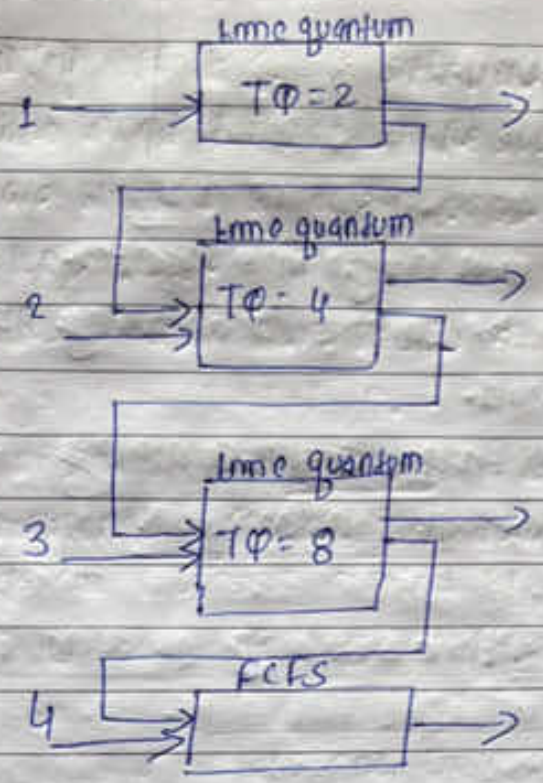
we devide process into thread. and execute multiple thread parallelly.

⊙ What is thread.
→ single sequence stream within a process.
→ independent path of execution in a process.
→ Light weight process.
→ used to achives parallelism by dividing a process task. which has independent path of execution.

E.g:- multiple tab in a browser. text editor (when you are typing in an editor, spell checking, formatting of text and saving the text are done concurrently by multiple threads.)

⊙ thread scheduling :- threads are scheduled for execution based on their priority.

⊙ thread context switching :-
→ OS save current state of thread and context switch to another thread of same process.
→ Fast switching.
→ doesn't include switching of memory address space.



any Process whose BT is less than 2 than schedule in First queue and exit. if BT more than 2 than it schedule in 2 queue and if more than 4 and less than 8 than schedule in 3rd queue. and if Greater than 8 than go in queue. in this way small job not go in starvation.

① Design of MLFQ:-

① No of queue.
② scheduling algo in each queue.
③ method to upgrade a process to a higher queue.
④ demote a process to lower priority.
⑤ any process is come → which queue will be pushed.

(2)

Each queue has its own scheduling algorithm.
Eg:- SP → RR , SP → RR BP → FCFS

→ scheduling among different sub-queues is implemented as Fixed Priority Preemptive.
→ If an interactive process comes and batch process is currently executing. then batch process will be preempted.
→ Problem: only after completion of all the process from the top level ready queue. the further level ready queue will be scheduled.

this came starvation for lower priority process.
→ convoy effect is present.

① Multi level feedback Queue scheduling :-
→ multiple sub-queues are presented.
→ allow the process to move b/w queues. the idea is to seprate process according to BT. If process uses too much cpu time, it will be moved to lower priority queue. this scheme leaves I/O bounds and interactive process in H.P q
→ A process that waits too much in a lower-priority queue may be moved to higher priority queue by ageing.

Advantages:-
① less starvation than MLQ.
② it's Flexible
③

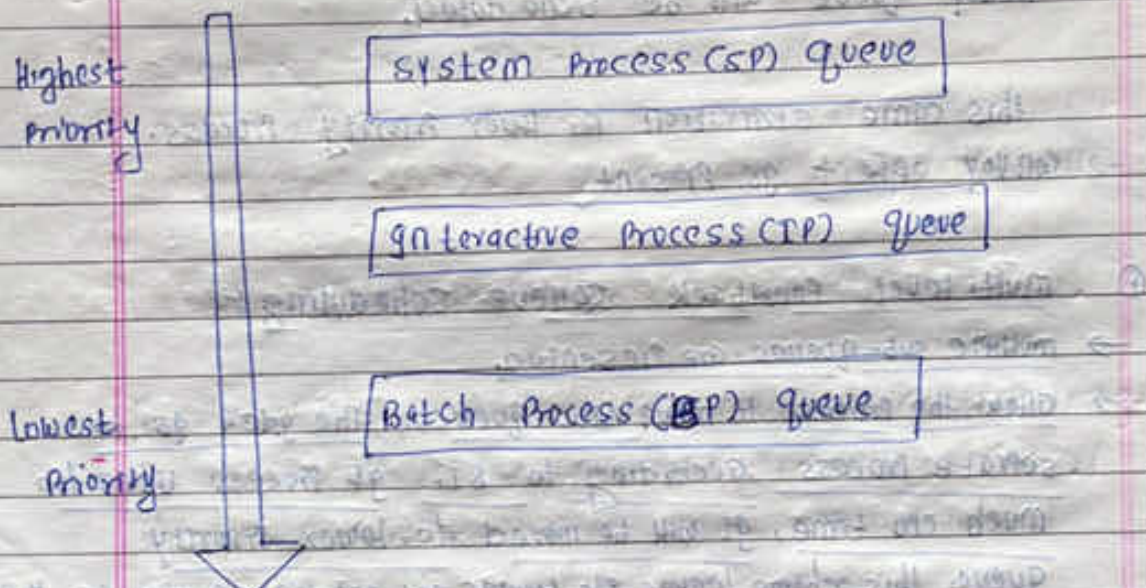
Page No.:
Date: / /

Convoy effect is a situation where many process who need to use a Resource for a short time are blocked by one process holding that Resource for a long time.
→ this cause poor Resource management.

Lec-14

① multilevel Queue scheduling :- (MLQ)

① system process : created by OS.



above are three type of Process.
→ Ready queue is devided in to three parts based on Priority.
→ a Process is Parmanently assign to one of the queue based on Property of Process.
① memory size ② Process Priority ③ Process type.
→ system Process created by o/s. highest Priority.
→ Interactive Process. (Needs user I/O) Foreground Process.
→ Batch Process (background process) Run silently no user input Required.